REGEX

cp.shewell@ulster.ac.uk

Patterns and Flags

- Regular expressions are patterns that provide a powerful way to search and replace in text.
- In JavaScript, they are available via the RegExp object, as well as being integrated in methods of strings.
- A regular expression (also “regexp”, or just “reg”) consists of a pattern and optional flags.

Syntax

- There are two syntaxes that can be used to create a regular expression object.
- The “long” syntax:

```
regexp = new RegExp("pattern", "flags");
```

- And the “short” one, using slashes "/":

```
regexp = /pattern/; // no flags
regexp = /pattern/gmi; // with flags g,m and i (to be covered soon)
```

- Slashes /.../ tell JavaScript that we are creating a regular expression. They play the same role as quotes for strings.

Syntax

- In both cases regexp becomes an instance of the built-in RegExp class.
- The main difference between these two syntaxes is that pattern using slashes /.../ does not allow for expressions to be inserted (like string template literals with ${...}). They are fully static.
- Slashes are used when we know the regular expression at the code writing time – and that's the most common situation.
- While new RegExp is more often used when we need to create a regexp "on the fly" from a dynamically generated string.

Flags

- Regular expressions may have flags that affect the search.
- There are only 6 of them in JavaScript:
- i
  - With this flag the search is case-insensitive: no difference between A and a
- g
  - With this flag the search looks for all matches, without it – only the first match is returned.
- m
  - Multiline mode.

Flags

- s
  - Enables “dotall” mode, that allows a dot . to match newline character \n
- u
  - Enables full Unicode support. The flag enables correct processing of surrogate pairs.
- y
  - “Sticky” mode: searching at the exact position in the text

str.match

- As mentioned previously, regular expressions are integrated with string methods.
- The method str.match(regexp) finds all matches of regexp in the string str.
- It has 3 working modes:

1. If the regular expression has flag g, it returns an array of all matches:

```
let str = "We will, we will rock you";

alert( str.match(/we/gi) ); // We,we (an array of 2 substrings that match)
```

- Please note that both We and we are found, because flag i makes the regular expression case-insensitive.

str.match

2. If there's no such flag it returns only the first match in the form of an array, with the full match at index 0 and some additional details in properties:

```
let str = "We will, we will rock you";

let result = str.match(/we/i); // without flag g

alert( result[0] );      // We (1st match)
alert( result.length ); // 1

// Details:
alert( result.index );  // 0 (position of the match)
alert( result.input );  // We will, we will rock you (source string)
```

- The array may have other indexes, besides 0 if a part of the regular expression is enclosed in parentheses.

str.match

3. And, finally, if there are no matches, null is returned (doesn't matter if there's flag g or not).

- This a very important nuance. If there are no matches, we don't receive an empty array, but instead receive null. Forgetting about that may lead to errors, e.g.:

```
let matches = "JavaScript".match(/HTML/); // = null

if (!matches.length) { // Error: Cannot read property 'length' of null
  alert("Error in the line above");
}
```

str.replace

- The method str.replace(regexp, replacement) replaces matches found using regexp in string str with replacement (all matches if there's flag g, otherwise, only the first one).

```
// no flag g
alert( "We will, we will".replace(/we/i, "I") ); // I will, we will

// with flag g
alert( "We will, we will".replace(/we/ig, "I") ); // I will, I will
```

- The second argument is the replacement string.

str.replace

- The second argument is the replacement string. We can use special character combinations in it to insert fragments of the match:

Symbols	Action in the replacement string
`$&`	inserts the whole match
`$` `	inserts a part of the string before the match
`$'`	inserts a part of the string after the match
`$n`	if `n` is a 1-2 digit number, then it inserts the contents of n-th parentheses, more about it in the chapter Capturing groups
`$<name>`	inserts the contents of the parentheses with the given `name`, more about it in the chapter Capturing groups
`$$`	inserts character `$`

- An example with $&:

```
alert( "I love HTML".replace(/HTML/, "$& and JavaScript") );
```

regexp.test

- The method regexp.test(str) looks for at least one match, if found, returns true, otherwise false.

```
let str = "I love JavaScript";
let regexp = /LOVE/i;

alert( regexp.test(str) ); // true
```

Review

- A regular expression consists of a pattern and optional flags: g, i, m, u, s, y.
- Without flags and special symbols (that we'll study later), the search by a regexp is the same as a substring search.
- The method str.match(regexp) looks for matches: all of them if there's g flag, otherwise, only the first one.
- The method str.replace(regexp, replacement) replaces matches found using regexp with replacement: all of them if there's g flag, otherwise only the first one.
- The method regexp.test(str) returns true if there's at least one match, otherwise, it returns false.

Character Classes

- Consider a practical task – we have a phone number like "+7(903)-123-45-67", and we need to turn it into pure numbers: 79031234567.
- To do so, we can find and remove anything that's not a number. Character classes can help with that.
- A *character class* is a special notation that matches any symbol from a certain set.
- For the start, let's explore the "digit" class. It's written as \d and corresponds to "any single digit".

Character Classes

- For instance, let's find the first digit in the phone number:

```
let str = "+7(903)-123-45-67";

let regexp = /\d/;

alert( str.match(regexp) ); // 7
```

- Without the flag g, the regular expression only looks for the first match, that is the first digit \d.

Character Classes

- Let’s add the g flag to find all digits:

```
let str = "+7(903)-123-45-67";

let regexp = /\d/g;

alert( str.match(regexp) ); // array of matches: 7,9,0,3,1,2,3,4,5,6,7

// let's make the digits-only phone number of them:
alert( str.match(regexp).join('') ); // 79031234567
```

- That was a character class for digits. There are other character classes as well.

Character Classes

- There are other character classes as well.
- \d ("d" is from "digit")
  - A digit: a character from 0 to 9.
- \s ("s" is from "space")
  - A space symbol: includes spaces, tabs \t, newlines \n and few other rare characters, such as \v, \f and \r.
- \w ("w" is from "word")
  - A "wordly" character: either a letter of Latin alphabet or a digit or an underscore _. Non-Latin letters (like cyrillic or hindi) do not belong to \w.
- For instance, \d\s\w means a "digit" followed by a "space character" followed by a "wordly character", such as 1 a.

Character Classes

- A regexp may contain both regular symbols and character classes.
- For instance, CSS\d matches a string CSS with a digit after it:

```
let str = "Is there CSS4?";
let regexp = /CSS\d/

alert( str.match(regexp) ); // CSS4
```

- Also, we can use many character classes:

```
alert( "I love HTML5!".match(/\s\w\w\w\w\d/) ); // ' HTML5'
```

- The match (each regexp character class has the corresponding result character):

```
                \s\w\w\w\w\d
I love HTML5
```

Inverse Classes

- For every character class there exists an “inverse class”, denoted with the same letter, but uppercased.
- The “inverse” means that it matches all other characters, for instance:
- \D
  - Non-digit: any character except \d, for instance a letter.
- \S
  - Non-space: any character except \s, for instance a letter.
- \W
  - Non-wordly character: anything but \w, e.g a non-latin letter or a space.

Inverse Classes

- In the beginning of the lecture, we saw how to make a number-only phone number from a string like +7(903)-123-45-67: find all digits and join them.

```
let str = "+7(903)-123-45-67";

alert( str.match(/\d/g).join('') ); // 79031234567
```

- An alternative, shorter way is to find non-digits \D and remove them from the string:

```
let str = "+7(903)-123-45-67";

alert( str.replace(/\D/g, "") ); // 79031234567
```

Dot

- A dot . is a special character class that matches “any character except a newline”.
- For instance:

```
alert( "Z".match(/./) ); // Z
```

- Or in the middle of a regexp:

```
let regexp = /CS.4/;

alert( "CSS4".match(regexp) ); // CSS4
alert( "CS-4".match(regexp) ); // CS-4
alert( "CS 4".match(regexp) ); // CS 4 (space is also a character)
```

- Please note that a dot means “any character”, but not the “absence of a character”.

Dot

- By default, a dot doesn't match the newline character \n.
- For instance, the regexp A.B matches A, and then B with any character between them, except a newline \n:

```
alert( "A\nB".match(/A.B/) ); // null (no match)
```

- There are many situations when we'd like a dot to mean literally "any character", newline included.
- That's what flag s does. If a regexp has it, then a dot . matches literally any character:

```
alert( "A\nB".match(/A.B/s) ); // A\nB (match!)
```

Spaces

- Usually, we pay little attention to spaces. For us strings 1-5 and 1 - 5 are nearly identical.
- But if a regexp doesn't take spaces into account, it may fail to work.
- Let's try to find digits separated by a hyphen:

```
alert( "1 - 5".match(/\d-\d/) ); // null, no match!
```

- Let's fix it adding spaces into the regexp \d - \d:

```
alert( "1 - 5".match(/\d - \d/) ); // 1 - 5, now it works
// or we can use \s class:
alert( "1 - 5".match(/\d\s-\s\d/) ); // 1 - 5, also works
```